

CPE 324-03: Advanced Digital Logic Design Laboratory

Lab 8 Video Display

Submitted by: Hannah Kelley

Date of Experiment: March 26 and 31, 2026 and April 2, 2026

Report Deadline: April 7, 2026

Demonstration Deadline: April 2, 2026

1 Introduction

This laboratory focuses on exploring computer display systems through practical implementation on the DE2-115 FPGA board. The objective is to design and implement digital logic capable of driving an SVGA display. Key aspects of the project include generating the necessary synchronization signals, mapping pixel addresses, and controlling color intensities for accurate image rendering. The culmination of the experiment is the visualization of a 256×256 pixel, 256-color representation of the DE2-115 board on the SVGA screen. Additionally, the system is designed to respond to user inputs from the board's buttons, allowing the displayed image to be resized and rotated dynamically.

2 Experiment Description

2.1 Part 1 Description

The first part of this lab focused on understanding the overall goal. To do so, the top-level design was considered as shown in Figure 1. The specifications of the SVGA monitor, specifically the pixel layout, can be found in Figure 2.

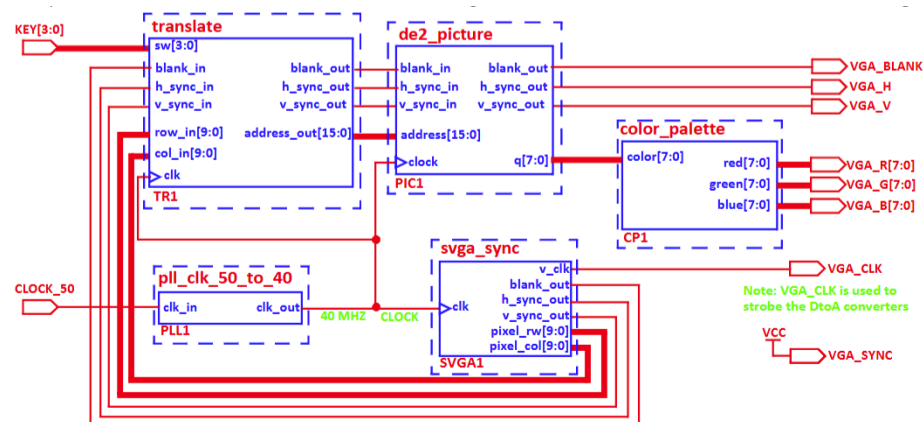


Figure 1: Display Driver Model Diagram

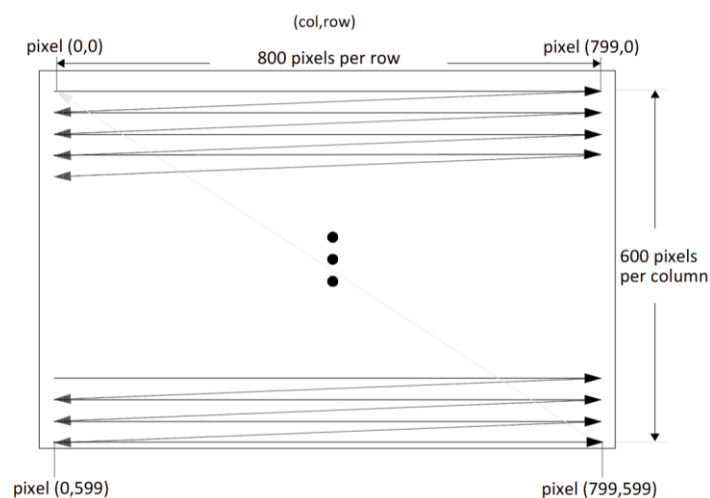


Figure 2: SVGA Pixel Layout

In Part A of this laboratory, the svga_sync module was completed to allow the correct signals to be sent to the SVGA display at the correct times to display an image. The specific signals Part A dealt with include the vertical and horizontal refresh cycles and synchronizations pulses. The timing diagrams used to construct these signals can be found in Figures 3 and 4.

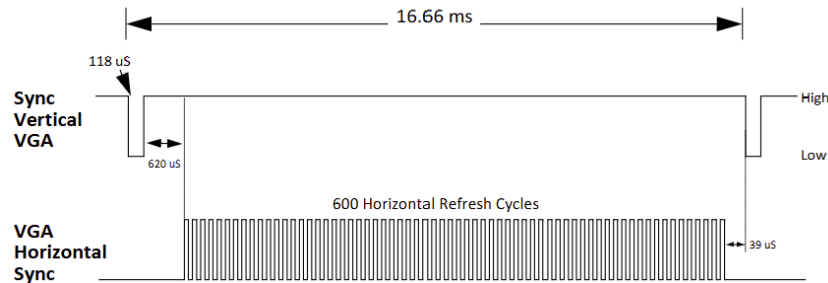


Figure 3: SVGA Vertical Refresh Cycle

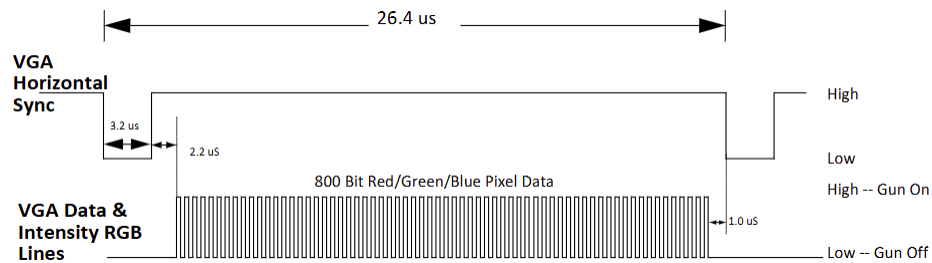


Figure 4: SVGA Horizontal Refresh Cycle

After calibrating the signals, the svga_sync module can be used to produce the expected output seen in Figure 5.

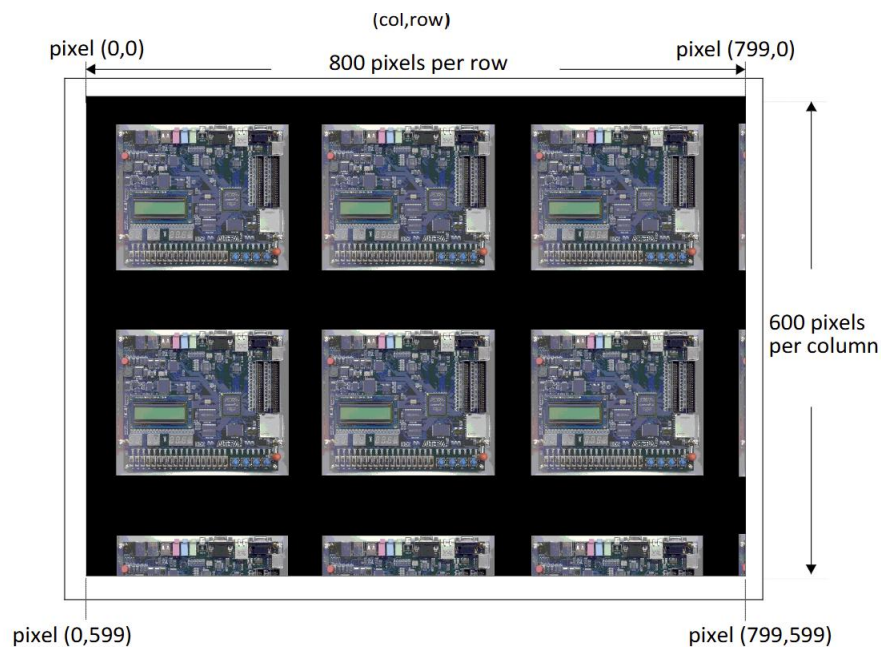


Figure 5: Part 1 Expected SVGA Display Output

2.2 Part 2 Description

After successfully displaying an image on the SVGA display in Part 1, the translate module was modified to display a single image in response to the inputs on keys 0-3 as shown in Table 1.

Key	Display
0	Scaled image (200%)
1	Original image
2	Rotated image (90° counter-clockwise)
3	Rotated image (180 °)

Table 1: Part 1 Key Assignments

The expected output is presented in Figure 6.

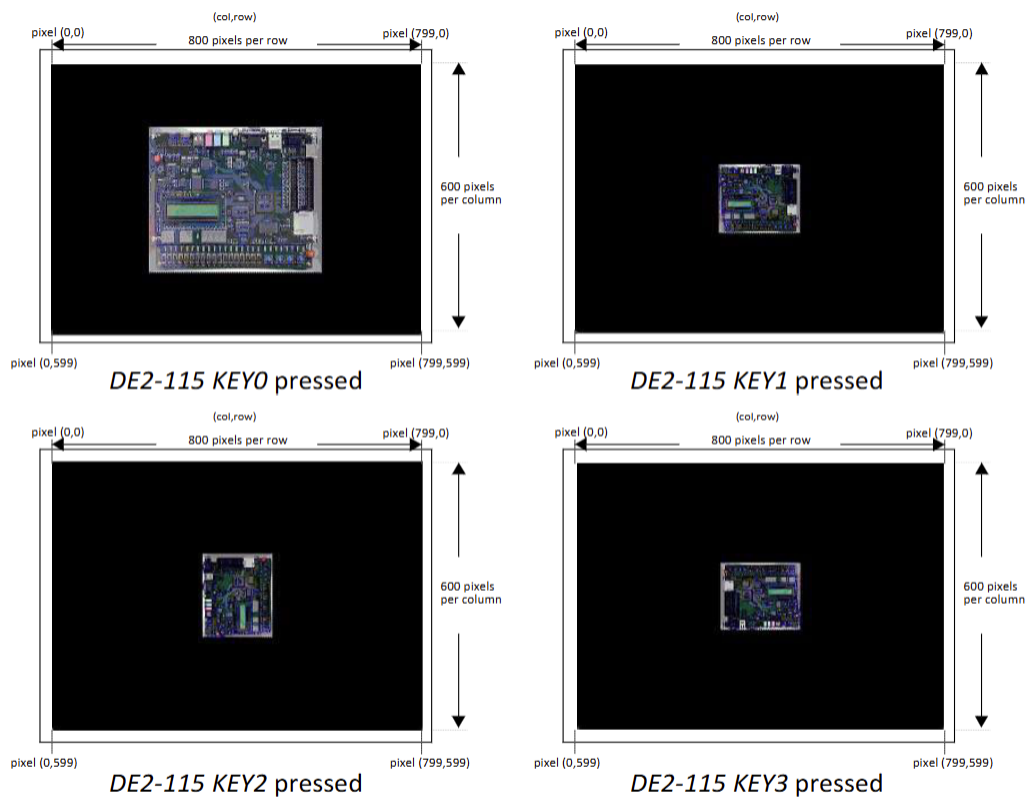


Figure 6: Part 2 Expected SVGA Display Output

3 Results

3.1 Part 1 Results

The values in Table 1 were calculated and implemented into the svga_sync module to obtain the SVGA display output in Figure 7.

Parameter	Value	Calculation
col_count max	1056	800 active pixels + 40 front porch + 128 sync pulse + 88 back porch
h_sync_out start	840	800 active pixels + 40 front porch
h_sync_out end	968	Sync start + sync pulse width
row_count max	631	600 active pixels + 1 front porch + 6 sync + 24 back porch
v_sync_out start	601	600 active lines + 1 front porch
v_sync_out end	606	Sync start + sync pulse width

Table 2: Timing Values and Calculations



Figure 7: Part 1 SVGA Display Output

3.2 Part B Results

Figures 8-11 show the various images displayed on the SVGA display based on the input from Keys 0-3.

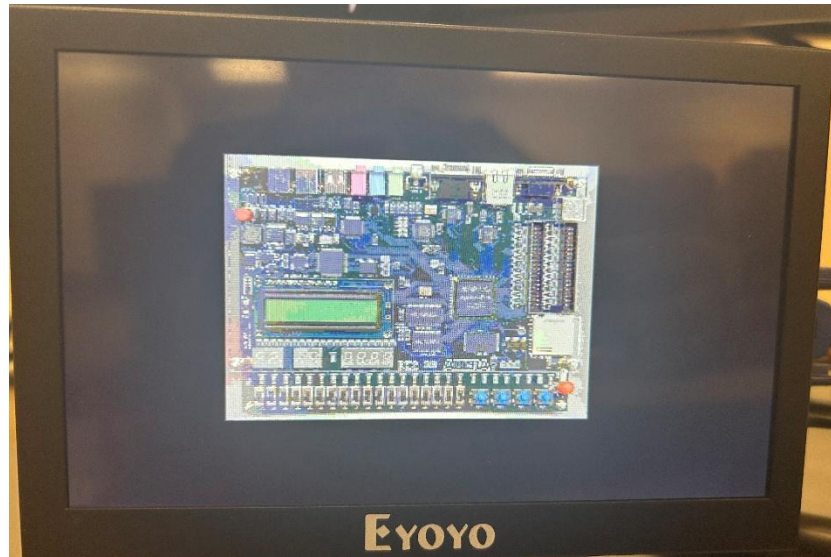


Figure 8: Scaled Image (200%)

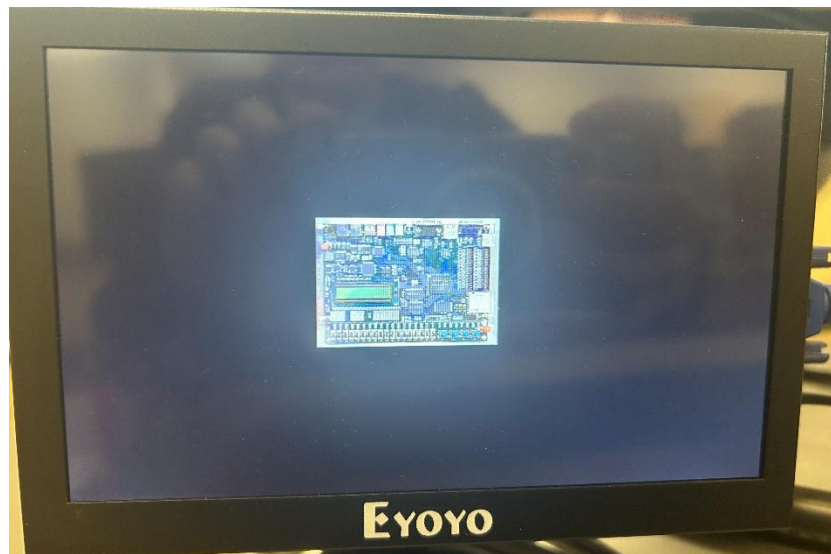


Figure 9: Original Image

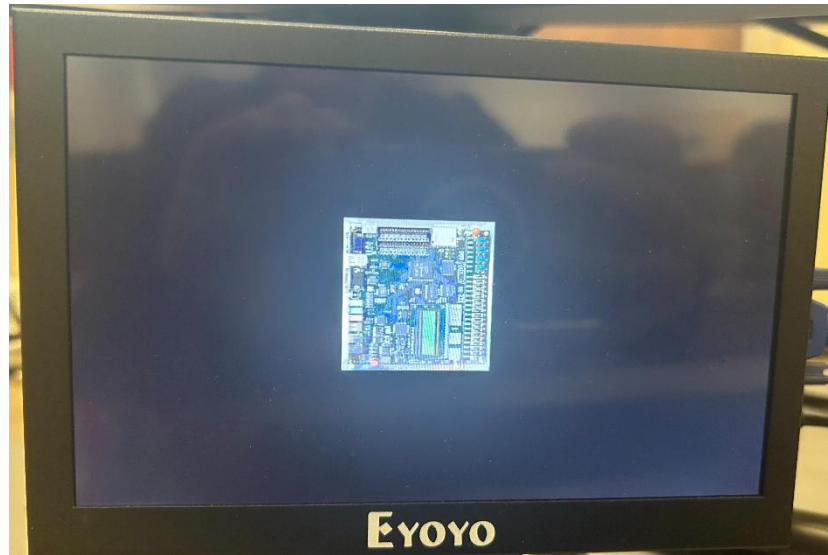


Figure 10: Rotate Image (90° counter-clockwise)

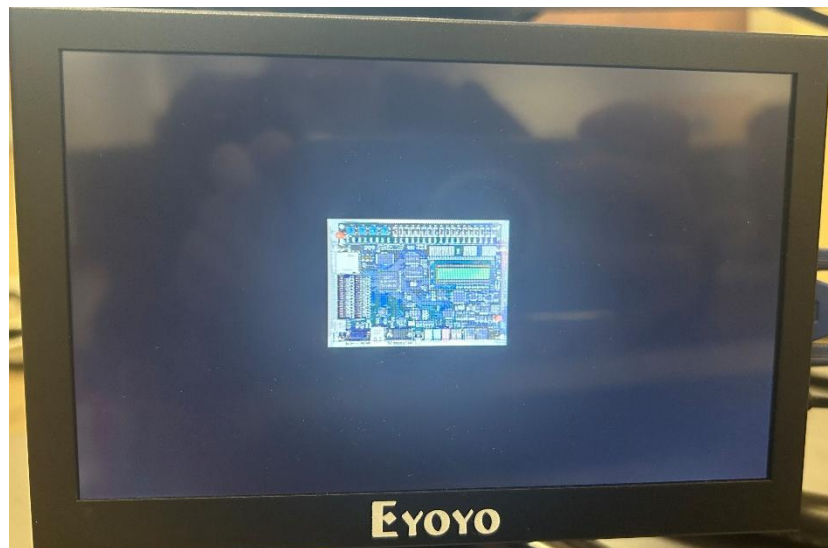


Figure 11: Rotate Image (180°)

3.4 Post-Lab Questions

After the laboratory, the following questions were considered:

- 1. What kind of compilation errors or warnings did you encounter? How did you correct the errors and how did you determine which warnings could be ignored?**
I did not encounter any errors aside from syntax issues caused by mismatched or missing begin and end statements. All warnings, including those related to clock signals and the number of processors, were safely ignored as they had no impact on the functionality of the design.
- 2. Explain briefly how a phase-lock-loop can be used to multiply and divide external clock frequencies that are used to drive logic within an FPGA such as the Cyclone IV E.**

A phase-locked loop achieves multiplication and division of external clock frequencies by regulating the voltage-controlled oscillator based on the phase difference between the input clock and the feedback signal. Once the loop stabilizes, the phase-locked loop locks the output frequency to a specific multiple or fraction of the input frequency. The phase detector measures the phase difference between the input clock and the feedback signal from the voltage-controlled oscillator, while the feedback divider scales the voltage-controlled oscillator output to create a feedback signal that is continuously compared to the input clock.

- 3. What is the purpose of the pll_clk_50_to_40 IP core module that was incorporated in the design? Is it possible to generate a 40 Mhz clock signal from a 50 Mhz one that is present on the DE2-115?**

The purpose of this module is to take the input clock frequency (40 MHz) and multiply it to obtain the output clock frequency of 50 MHz. It may be possible to create a 40 MHz clock frequency from a 27 MHz clock, though it would be difficult since the two frequencies do not have convenient common factors.

- 4. If the clock that was used on the DE2-115 was a 27 Mhz clock instead of a 50 Mhz one then would it be possible to create a perfectly accurate 40 Mhz clock assuming that the 27 Mhz clock was perfectly accurate and the same PLL hardware is being used. If this is not possible explain why not and how close to 27 Mhz you could get.**

If the DE2-115 used a 27 MHz clock instead of 50 MHz, it would not be possible to generate a perfectly accurate 40 MHz clock using the same PLL hardware. This is because 40 MHz is not an integer multiple or simple fractional ratio of 27 MHz, which prevents the PLL from locking exactly to 40 MHz. The closest achievable frequency would depend on the allowable multiplication and division factors of the PLL, but it would only approximate 40 MHz rather than match it precisely.

- 5. What is the purpose and function of the de2_picture IP core module? From the reports generated by the Quartus compilation/synthesis/map/place/route process what are the type and number of FPGA internal resources that were used for this module? For the memory element(s) that were employed in the module did Quartus treat them as a combinational ROM type element or as a synchronous Block ROM? Explain your answer?**

The de2_picture IP core module serves to store a 256-color bitmap of the DE2-115 image ROM. The upper 8 bits of the address correspond to the row, while the lower 8 bits correspond to the column. Pixel color data, ranging from 0 to 255, is read from a text file and loaded into memory using a directive, resulting in a 65,536-element array of 8-bit registers. The FPGA resources for this module are typically implemented as a synchronous Block ROM, since data storage and retrieval occur in sync with the clock signal.

- 6. What is the purpose and function of the color_palette IP core module? From the reports generated by the compilation/synthesis/map/place/route process what are**

**the type and number of FPGA internal resources that were used for this module?
For the memory element(s) that were employed in the module did Quartus treat them as a combinational ROM type element or as a synchronous Block ROM?**

Explain your answer?

The color_palette IP module exists to convert 8-bit color values into the corresponding RGB values used to drive the SVGA display. The three color palettes are characterized by 256 elements, each 8-bits in size, that describe the possible colors at various intensity levels. These arrays are implemented as combinational ROM, since the output color values update immediately when the input changes, independent of the clock signal.

4 Conclusion

In conclusion, this laboratory successfully demonstrated the practical implementation of a computer display system using the DE2-115 FPGA board. By designing the necessary digital logic for SVGA output, the experiment reinforced key concepts such as synchronization signal generation, pixel addressing, and color control. The ability to render a 256×256 pixel, 256-color image and incorporate interactive features like resizing and rotation highlighted both the flexibility and power of FPGA-based design. Overall, the lab provided valuable insight into how hardware-level design directly impacts visual output in display systems.

One of the most challenging aspects of this lab was accurately calculating values based on the interpretation of the timing diagram. Translating the diagram into precise numerical values for synchronization intervals, pixel counts, and control signals required careful analysis and attention to detail. Small misinterpretations often led to incorrect display behavior, making debugging both time-consuming and iterative. This difficulty ultimately reinforced the importance of thoroughly understanding timing relationships and how they translate into hardware logic, making it a valuable learning experience despite the challenges.

Appendix A – Complete lab.v File

```
// CPE 324 Video Driver Template File -- Laboratory Assignment 8
// (c) B. Earl Wells, University of Alabama in Huntsville 3/25/2026

// Top-level SVGA DE2-115 Display Driver
module lab8(
    input CLOCK_50,
    input [3:0] KEY,
    output VGA_SYNC, VGA_BLANK, VGA_CLK, VGA_H, VGA_V,
    output [7:0] VGA_B, VGA_G, VGA_R
);

    wire clock_40;
    wire n0,n1,n2,n3,n4,n5;
    wire [15:0] b0;
    wire [9:0] b1,b2;
```

```

wire [7:0] b3;

assign VGA_SYNC = 1'b1;

color_palette CP1(
    .color(b3),
    .blue(VGA_B),
    .green(VGA_G),
    .red(VGA_R)
);

de2_picture PIC1(
    .blank_in(n3),
    .h_sync_in(n4),
    .v_sync_in(n5),
    .address(b0),
    .clock(clock_40),
    .blank_out(VGA_BLANK),
    .h_sync_out(VGA_H),
    .v_sync_out(VGA_V),
    .q(b3)
);

pll_clk_50_to_40 PLL1(
    .clk_in(CLOCK_50),
    .clk_out(clock_40)
);

svga_sync SVGA1(
    .clk(clock_40),
    .v_clk(VGA_CLK),
    .blank_out(n0),
    .h_sync_out(n1),
    .v_sync_out(n2),
    .pixel_rw(b1),
    .pixel_col(b2)
);

translate TR1(
    .sw(KEY),
    .blank_in(n0),
    .h_sync_in(n1),
    .v_sync_in(n2),
    .col_in(b2),
    .row_in(b1),
    .clk(clock_40),

```

```
.blank_out(n3),  
.h_sync_out(n4),  
.v_sync_out(n5),  
.address_out(b0)  
);
```

```
endmodule
```

```
// SVGA sync module
```

```
module svga_sync(  
    input clk,  
    output v_clk,  
    output blank_out,  
    output reg h_sync_out,  
    output reg v_sync_out,  
    output reg [9:0] pixel_rw,  
    output reg [9:0] pixel_col  
);
```

```
reg video_on_h, video_on_v;
```

```
always @(posedge clk) begin: main_loop
```

```
    reg [10:0] col_count = 0;  
    reg [9:0] row_count = 0;
```

```
    if (col_count == 1056)  
        col_count <= 0;  
    else  
        col_count <= col_count + 1;
```

```
    if (col_count >= 840 && col_count <= 968)  
        h_sync_out <= 0;  
    else  
        h_sync_out <= 1;
```

```
    if (row_count == 631 && col_count == 1056)  
        row_count <= 0;  
    else if (col_count == 1056)  
        row_count <= row_count + 1;
```

```
    if (row_count >= 601 && row_count <= 606)  
        v_sync_out <= 0;  
    else  
        v_sync_out <= 1;
```

```
    if (col_count < 800)
```

```

        begin
            video_on_h <= 1;
            pixel_col <= col_count;
        end
    else
        video_on_h <= 0;

        if (row_count < 600)
            begin
                video_on_v <= 1;
                pixel_rw <= row_count;
            end
        else
            video_on_v <= 0;
        end

        assign blank_out = video_on_h && video_on_v;
        assign v_clk = ~clk;

```

```
endmodule
```

```
// Translate module
```

```

module translate(
    input [3:0] sw,
    input blank_in,
    input h_sync_in,
    input v_sync_in,
    input [9:0] row_in,
    input [9:0] col_in,
    input clk,
    output blank_out,
    output reg h_sync_out,
    output reg v_sync_out,
    output reg [15:0] address_out
);

```

```

    reg blank_out_translate = 1;
    reg [1:0] video_mode = 1;

```

```

    always @(negedge clk) begin
        if (sw[0] == 0) video_mode = 0;
        else if (sw[1] == 0) video_mode = 1;
        else if (sw[2] == 0) video_mode = 2;
        else if (sw[3] == 0) video_mode = 3;
    end

```

```

always @(posedge clk) begin
    h_sync_out = h_sync_in;
    v_sync_out = v_sync_in;
end

assign blank_out = blank_out_translate & blank_in;

always @(posedge clk) begin: mainloop
    reg [9:0] row_out, col_out;
    address_out = {row_in[7:0], col_in[7:0]};
    blank_out_translate = 1;

    case (video_mode)
        0: begin
            if(row_in[9:1] >= 22 && row_in[9:1] < 278 &&
               col_in[9:1] >= 72 && col_in[9:1] < 328) begin
                row_out = (row_in - 44)/2;
                col_out = (col_in - 144)/2;
                address_out = {row_out[7:0], col_out[7:0]};
                blank_out_translate = 1;
            end else blank_out_translate = 0;
        end
        1: begin
            if(row_in >= 172 && row_in < 428 &&
               col_in >= 272 && col_in < 528) begin
                row_out = row_in - 172;
                col_out = col_in - 272;
                address_out = {row_out[7:0], col_out[7:0]};
                blank_out_translate = 1;
            end else blank_out_translate = 0;
        end
        2: begin
            if(row_in >= 172 && row_in < 428 &&
               col_in >= 272 && col_in < 528) begin
                row_out = col_in - 272;
                col_out = 172 - row_in;
                address_out = {row_out[7:0], col_out[7:0]};
                blank_out_translate = 1;
            end else blank_out_translate = 0;
        end
        3: begin
            if(row_in >= 172 && row_in < 428 &&
               col_in > 272 && col_in < 528) begin
                row_out = 172 - row_in;
                col_out = 272 - col_in;
                address_out = {row_out[7:0], col_out[7:0]};
            end
        end
    endcase
end

```

```
        blank_out_translate = 1;
    end else blank_out_translate = 0;
    end
endcase
end
```

```
endmodule
```

```
// PLL Clock Converter
```

```
module pll_clk_50_to_40(input clk_in, output clk_out);
```

```
    wire [4:0] clockgrp;
```

```
    assign clk_out = clockgrp[0:0];
```

```
    altpll altpll_component (
```

```
        .inclk ({1'b0, clk_in}),
```

```
        .clk (clockgrp),
```

```
        .activeclock (),
```

```
        .areset (0),
```

```
        .clkbad (),
```

```
        .clkena ({6{1'b1}}),
```

```
        .clkloss (),
```

```
        .clkswitch (0),
```

```
        .configupdate (0),
```

```
        .enable0 (),
```

```
        .enable1 (),
```

```
        .extclk (),
```

```
        .extclkena ({4{1'b1}}),
```

```
        .fbn (1'b1),
```

```
        .fbmimicbidir (),
```

```
        .fbout (),
```

```
        .fref (),
```

```
        .icdrclk (),
```

```
        .locked (),
```

```
        .pfdena (1'b1),
```

```
        .phasecounterselect ({4{1'b1}}),
```

```
        .phasedone (),
```

```
        .phasestep (1'b1),
```

```
        .phaseupdown (1'b1),
```

```
        .pllenna (1'b1),
```

```
        .scanaclr (0),
```

```
        .scanclk (0),
```

```
        .scanclkena (1'b1),
```

```
        .scandata (0),
```

```
        .scandataout (),
```

```
        .scandone (),
```

```
        .scanread (0),
```

```

        .scanwrite (0),
        .sclkout0 (),
        .sclkout1 (),
        .vcooverrange (),
        .vcounderrange ()
    );
    defparam
        altpll_component.clk0_divide_by = 5,
        altpll_component.clk0_multiply_by = 4,
        altpll_component.inclk0_input_frequency = 20000,
        altpll_component.intended_device_family = "Cyclone IV E";
endmodule

// Color Palette Module
module color_palette(
    input [7:0] color,
    output reg [7:0] red, green, blue
);
    reg [7:0] red_palette [0:255], green_palette[0:255], blue_palette[0:255];
    initial begin
        red_palette[0] = 8'd0;
        red_palette[1] = 8'd128;
        red_palette[2] = 8'd0;
        red_palette[3] = 8'd128;
        // ... continue initializing all 256 palette values
    end
endmodule

```